KernelForge: Measured LLM-Generated Triton Kernels

Imanol Armando González Solís, José Manuel Martínez Morales, Pablo Esteban Reyes Herrera
Sebastian Gerritsen Ortiz, Victor Javier Quintana Cisneros

Tecnológico de Monterrey — TC3002B (ACA-2026)

Tecnológico de Monterrey

Abstract

KernelForge studies when LLM-generated Triton GPU kernels are worth writing, and whether grammar/planner interventions improve over plain Gemma 4 code generation. The project combines TritonBench-T [1] prompt and ledger tooling, grammar-constrained decoding, a lightweight semantic checker, and KAGBench: our project-created benchmark adapter for agentic kernel workflows. The measured result is deliberately mixed: generated Triton kernels can be much faster than PyTorch references on favorable memory-bound and fusion-friendly operators, with a verified 11.25× KAGBench speedup, but our constrained/planner variants do not show a statistically reliable correctness improvement over no-grammar or plain Gemma 4 baselines. The reusable artifact is the evidence loop: prompts, candidates, repairs, correctness, timing, and caveats are all preserved as JSONL traces for later fine-tuning or preference-data experiments.

1. Problem statement

- **Pain point:** GPU researchers and library authors spend repeated effort turning tensor math into correct, masked, launchable Triton kernels.
- **Current baseline:** Raw LLM completions often look plausible but fail through bad APIs, missing masks, wrapper mismatches, or numerical errors.
- **Our research question:** Can grammar constraints or planner–implementer decomposition improve over plain Gemma 4, and which generated kernels actually beat PyTorch?

2. Background

A context-free grammar describes which token sequences belong to a language. For Triton, grammar rules are a practical middle layer: stricter than natural language prompting, but lighter than a full compiler. XGrammar uses those rules during decoding so the model is guided toward legal Triton JIT blocks. Triton [2] is a good target because it exposes GPU programming through Python-like kernels while still compiling to efficient device code.

- **Formal grammars:** BNF/GBNF rules define the allowed shape of Triton JIT functions.
- **Constrained decoding:** The model is restricted to grammar-valid continuations while generating code.
- **Triton DSL:** A Python-embedded language for writing GPU kernels with explicit blocks, masks, and program IDs.

3. Our differentiator

One-line differentiator: KernelForge is a measured generation loop, not only a demo generator: it tests plain Gemma 4 against constrained and planner–implementer variants, then keeps reusable traces for future tuning even when the measured answer is *no correctness improvement*.

- **C1.** The planner emits explicit sections for pattern, API contract, program mapping, numerics, non-default choices, and risks before code is written.
- **C2.** The implementer receives the plan plus hard Triton rules, optional grammar constraints, and public-test context.
- **C3.** Plans, candidates, failures, repair directives, paired correctness, and speed measurements form a base for future fine-tuning, distillation, or preference ranking.

4. System architecture

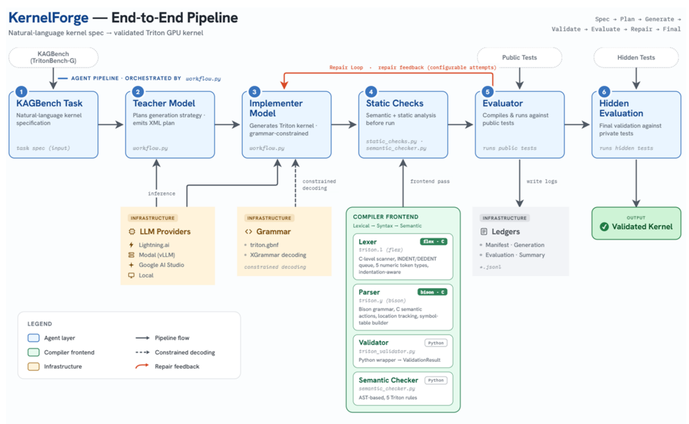

Figure 1. KernelForge end-to-end pipeline: from kernel specification to checked candidate, GPU evaluation, repair feedback, and reusable ledgers.

KAGBench (Kernel Agentic Bench) is a 48-task dataset created inside this project to make TritonBench-derived cases fit an agentic loop. Each case is a self-contained folder with a prompt, PyTorch reference, public tests, hidden tests, tags, and source provenance. A teacher/planner writes a compact strategy; an implementer emits a Triton candidate; static checks, GPU evaluation, optional repair prompts, and JSONL ledgers preserve the full trace for benchmarking and later training data.

5. Grammar design

- **L1 — Skeleton:** `@triton.jit` decorators, Python function headers, parameters, and bounded indentation.
- **L2 — Declarations:** annotated/default parameters, including `BLOCK_SIZE: tl.constexpr` and local assignments.
- **L3 — Expressions:** arithmetic, comparisons, tuple/list expressions, indexing, and qualified calls such as `tl.load`.
- **L4 — Control flow:** bounded `if`, `for`, `while`, and `with` blocks inside Triton JIT bodies.

The grammar intentionally focuses on Triton JIT blocks instead of arbitrary Python modules. Imports, benchmark wrappers, and test harness code stay outside the constrained region because the benchmark adapters need flexible signatures and glue code. The active artifact is `grammar/triton.gbnf`; current tests smoke-compile it through XGrammar and check representative accept/reject snippets. It is still an experiment, not a complete Triton parser.

6. Generation pipeline

1. **Prompt construction:** Build a prompt from a KAGBench case derived from TritonBench-T, including the task description, expected wrapper behavior, PyTorch reference, and optional public tests.
2. **Planner stage:** Ask a planning model for the kernel pattern, API contract, program mapping, numerics, and likely risks without writing code.
3. **Implementer stage:** Generate code from the plan at temperature 0; when enabled, pass `grammar/triton.gbnf` to vLLM with the `xgrammar` backend.
4. **Static triage:** Reject empty or unparsable outputs for evaluation, and log semantic warnings for missing masks, decorators, program IDs, or suspicious PyTorch calls.
5. **Compile & run:** Evaluate generated code against public or hidden tests with a local subprocess or Modal GPU backend.
6. **Repair/data loop:** When public evaluation fails, feed the error summary back to the planner/implementer loop and preserve every attempt as data for analysis or future tuning.

7. Experimental setup

- **Benchmark data:** TritonBench-T simple set (166 tasks) for raw/LLGuidance baselines; KAGBench, our 48-task project-created adapter from TritonBench-G/T cases, for planner–implementer and repair experiments.
- **Hardware:** Modal L4 for the Gemma 4 vLLM backend; local ROCm/Triton subprocesses for KAGBench candidate evaluation and timing.
- **Software:** Python/uv workflow, PyTorch + Triton evaluation environment, vLLM structured outputs, llama.cpp/LLGuidance GBNF constraints, and JSONL ledgers.
- **Repetitions:** Correctness-first evaluation per candidate; speed timing on hidden-passing candidates with `triton.testing.do_bench`. Top KAGBench speedups were retested with warmup=100, rep=1000, 5 repeats.
- **Metrics/data:** call@1, exe@1, public/hidden pass, mismatch summaries, semantic warnings, truncation, latency, planner text, candidate code, repair traces, and speedup over PyTorch references.

8. Results

Headline result: generated Triton can be **11.25×** faster than the PyTorch reference on a favorable KAGBench embedding task, but **our completed paired comparisons show no reliable correctness gain from constraints or planning**. That negative result is part of the evidence: the ledgers let us separate real speedups from generation-method claims.

Method	Scope	Pass rate	Median	Paired result
Plain Gemma 4 vLLM, no grammar	TritonBench-T (166)	call@1 40/166; exe@1 21/166	1.01×	Baseline
Gemma 4 + LLGuidance/GBNF	TritonBench-T (166)	call@1 32/166; exe@1 21/166	1.00×	exe@1 tie; $p = 1.000$
Planner + GBNF constrained	KAGBench (48)	public 25/48; hidden 16/48	1.10×	no gain; $p = 1.000$
Planner, no grammar	KAGBench (48)	public 22/48; hidden 17/48	1.09×	KAGBench baseline

Table 1. Completed runs show no statistically reliable correctness gain from constraints or planner–implementer decomposition, while correctness-passing Triton kernels can still provide real speedups over PyTorch references.

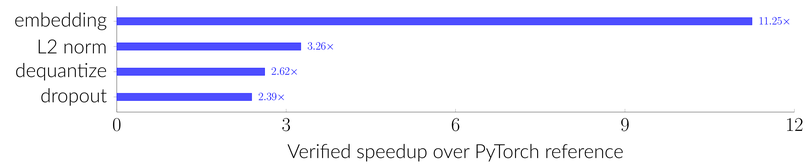

Figure 2. How generated kernels produced speedup: the best candidates used direct Triton program mapping, masked block loads/stores, in-kernel reductions or simple fused arithmetic, and avoided PyTorch dispatch/intermediate tensors. These wins are operator-dependent; matrix-multiply-like cases often lose to vendor libraries.

9. Contributions & limitations

- **Reusable contribution:** KAGBench plus the ledgers provide a base for an agentic improvement loop: passing kernels become supervised examples, failed repairs become edit data, and multiple candidates can become preference/ranking data.
- **Main Triton advantage:** Memory-bound or fusion-friendly tasks can use one custom kernel to avoid PyTorch dispatch, intermediate tensors, and implicit dtype/masking choices.
- **What changed over plain Gemma 4?** Not correctness so far: LLGuidance tied plain Gemma 4 on TritonBench-T exe@1, and KAGBench constrained/no-grammar hidden-pass differences were not significant.
- **How models generated speedup:** Winning candidates used one-program-per-row/block mappings, `tl.arange` offsets, masked `tl.load`/`tl.store`, in-kernel reductions, and fused elementwise arithmetic.
- **Caveats:** Local LLGuidance was slower than Modal vLLM plain Gemma 4, complete KAGBench constraints used llama.cpp GBNF, and speedups are measured only on correctness-passing candidates.

References

[1] J. Li *et al.*, "TritonBench: Benchmarking large language model capabilities for generating Triton operators," in *Findings of ACL*, 2025, pp. 23053–23066, doi: 10.18653/v1/2025.findings-acl.1183.

[2] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. MAPL*, 2019, pp. 10–19, doi: 10.1145/3315508.3329973.